

در بخش دوم نیز ابتدا فایل دیتاست رو که مربوط به spotify 1 million tracks است و دارای فرمت CSV. می باشد ، با دستور معمول پای اسپارک می خوانیم و نمایش می دهیم :

```
# Import
from pyspark.sql import SparkSession

# Create SparkSession
spark = SparkSession.builder.getOrCreate()
# Read CSV File
spotifydf = spark.read.csv("/content/gdrive/MyDrive/spotify/spotify_data.csv", header=True, inferSchema=True)
```

_c0	artist_name	track_name	track_id	popularity	year	genre	danceability	energy	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	duration_ms	time_signature
0	Jason Mraz	I Won't Give Up	530F56c32A9RTuWZ...	68	2012	acoustic	0.483	0.303	4	-10.058	1	0.0429	0.694	0.0	0.115	0.139	133.406	240166.0	3.0
1	Jason Mraz	93 Million Miles	1s8tP3jP4GZcyHDs...	50	2012	acoustic	0.572	0.454	3	-10.286	1	0.0258	0.477	1.37e-05	0.0974	0.515	140.182	216387.0	4.0
2	Joshua Hyslop	Do Not Let Me Go	78RCa8PMyuvr2VU3...	57	2012	acoustic	0.409	0.234	3	-13.711	1	0.0323	0.338	5e-05	0.0895	0.145	139.832	158960.0	4.0
3	Boyce Avenue	Fast Car	63ws2UhlZLh10syr...	58	2012	acoustic	0.392	0.251	10	-9.845	1	0.0363	0.807	0.0	0.0797	0.508	204.961	304293.0	4.0
4	Andrew Belle	Sky's Still Blue	6nIYC1v3JAFi6uLi...	54	2012	acoustic	0.43	0.791	6	-5.419	0	0.0302	0.0726	0.0193	0.11	0.217	171.864	244320.0	4.0
5	Chris Smither	What They Say	24NvptbNKGs6sPyIV...	48	2012	acoustic	0.566	0.57	2	-6.42	1	0.0329	0.688	1.73e-06	0.0943	0.96	83.403	166240.0	4.0
6	Matt Wertz	Walking in a Wint...	0BP7HsVL6G3URGrEv...	48	2012	acoustic	0.575	0.606	9	-8.197	1	0.03	0.0119	0.0	0.0675	0.364	121.083	152307.0	4.0
7	Green River Ordin...	Dancing Shoes	3Y6BuzQCg9p4yH347...	45	2012	acoustic	0.586	0.423	7	-7.459	1	0.0261	0.252	5.83e-06	0.0976	0.318	138.133	232373.0	4.0
8	Jason Mraz	Living in the Moment	3ce7k1L4EkZppZp1...	44	2012	acoustic	0.65	0.628	7	-7.16	1	0.0232	0.0483	0.0	0.119	0.7	84.141	235080.0	4.0
9	Boyce Avenue	Heaven	2EKxmyMudAVX1aHCn...	58	2012	acoustic	0.619	0.28	8	-10.238	0	0.0317	0.73	0.0	0.103	0.292	129.948	250063.0	4.0
10	Tristan Prettyman	Say Anything	1RjEDh1p2L30pGL...	45	2012	acoustic	0.657	0.43	1	-10.202	1	0.0314	0.534	0.000238	0.12	0.335	91.067	236379.0	4.0
11	David Gray	Money (That's Wha...	38s5J12az76M4c7a...	45	2012	acoustic	0.655	0.692	5	-6.217	1	0.0326	0.568	2.34e-05	0.119	0.547	121.455	234270.0	4.0
12	Boyce Avenue	Someone Like You	6VtoP2s3t5oCmPQI...	55	2012	acoustic	0.439	0.207	1	-9.573	1	0.0297	0.608	0.0	0.186	0.264	136.514	276147.0	4.0
13	Jason Mraz	The Woman I Love	0AYG9AcwEqgAw30...	40	2012	acoustic	0.591	0.647	4	-8.34	1	0.0277	0.067	6.57e-05	0.231	0.678	79.68	190752.0	4.0
14	Eddie Vedder	I Shall Be Released	0J11fISejodq8Iy0Q...	44	2012	acoustic	0.45	0.713	6	-7.503	0	0.0386	0.157	0.0	0.992	0.36	74.71	283587.0	4.0
15	The Civil Wars	Kingdom Come	1I2c5f9u1w7Ge2eOp...	41	2012	acoustic	0.497	0.277	4	-11.382	0	0.0373	0.846	4.77e-06	0.111	0.179	81.367	222560.0	4.0
16	Gabrielle Aplin	Home	20kpTTSv14ok6m1u...	57	2012	acoustic	0.439	0.265	2	-10.72	1	0.0343	0.736	0.0	0.093	0.382	140.494	247002.0	4.0
17	Harley Poe	Transvestities Ca...	146KacVTWMyX50jf...	39	2012	acoustic	0.499	0.614	7	-8.175	1	0.0621	0.151	3.52e-06	0.0555	0.605	89.038	276973.0	4.0
18	Ron Pope	One Grain of Sand	3yqJGfvXtPZL1uHVe...	49	2012	acoustic	0.713	0.824	3	-7.168	1	0.0393	0.106	8.03e-05	0.13	0.696	128.028	207240.0	4.0
19	Sara Bareilles	Once Upon Another...	17KG9zr1C61P8f1CN1...	39	2012	acoustic	0.275	0.216	2	-14.504	1	0.0493	0.896	0.0	0.231	0.0551	95.421	324333.0	5.0

بعد از پرینت گرفتن تایپ یا نوع فرمت داده های هر ستون ، متوجه شدیم ستون هایی که داده های عددی دارند به اشتباه دارای تایپ string هستند :

```
spotifydf.printSchema()

root
|-- _c0: integer (nullable = true)
|-- artist_name: string (nullable = true)
|-- track_name: string (nullable = true)
|-- track_id: string (nullable = true)
|-- popularity: string (nullable = true)
|-- year: string (nullable = true)
|-- genre: string (nullable = true)
|-- danceability: string (nullable = true)
|-- energy: string (nullable = true)
|-- key: string (nullable = true)
|-- loudness: string (nullable = true)
|-- mode: string (nullable = true)
|-- speechiness: string (nullable = true)
|-- acousticness: string (nullable = true)
|-- instrumentalness: string (nullable = true)
|-- liveness: string (nullable = true)
|-- valence: double (nullable = true)
|-- tempo: double (nullable = true)
|-- duration_ms: double (nullable = true)
|-- time_signature: double (nullable = true)
```

برای حل این مشکل به صورت دستی تمام ستون هایی که تایپ آنها اشتباه بودند را اصلاح کردیم که به عنوان نمونه یکی از آنها را در زیر مشاهده می کنید :

```

from pyspark.sql.functions import *
from pyspark.sql.types import *

new_spotifydf=spotifydf_no_genre.withColumn('popularity', spotifydf['popularity'].cast(DoubleType()))

```

این کد را برای تمام ستون های اشتباه تکرار می کنیم تا نتیجه ی درست حاصل شود :

```

[ ] new_spotifydf.printSchema()

root
 |-- _c0: integer (nullable = true)
 |-- artist_name: string (nullable = true)
 |-- track_name: string (nullable = true)
 |-- track_id: string (nullable = true)
 |-- popularity: double (nullable = true)
 |-- year: integer (nullable = true)
 |-- danceability: double (nullable = true)
 |-- energy: double (nullable = true)
 |-- key: integer (nullable = true)
 |-- loudness: double (nullable = true)
 |-- mode: integer (nullable = true)
 |-- speechiness: double (nullable = true)
 |-- acousticness: double (nullable = true)
 |-- instrumentalness: double (nullable = true)
 |-- liveness: double (nullable = true)
 |-- valence: double (nullable = true)
 |-- tempo: double (nullable = true)
 |-- duration_ms: double (nullable = true)
 |-- time_signature: double (nullable = true)

```

در مرحله ی بعد برای فرستادن داده های دیتاستمون به الگوریتم خوشه بندی ، تمام ستون هارا به بردار های feature تبدیل می کنیم :

```

[8] from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.clustering import KMeans
import matplotlib.pyplot as plt

# تبدیل داده ها به فرمت وکتور
assembler = VectorAssembler(inputCols=new_spotifydf.columns, outputCol="features")
spotifydf_features = assembler.transform(new_spotifydf)

```

_c0	popularity	year	danceability	energy	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	duration_ms	time_signature	features
0	68.0	2012	0.483	0.303	4	-10.058	1	0.0429	0.694	0.0	0.115	0.139	133.406	240166.0	3.0	[0.0,68.0,2012.0,...]
1	50.0	2012	0.572	0.454	3	-10.286	1	0.0258	0.477	1.37E-5	0.0974	0.515	140.182	216387.0	4.0	[1.0,50.0,2012.0,...]
2	57.0	2012	0.409	0.234	3	-13.711	1	0.0323	0.338	5.0E-5	0.0895	0.145	139.832	158960.0	4.0	[2.0,57.0,2012.0,...]
3	58.0	2012	0.392	0.251	10	-9.845	1	0.0363	0.807	0.0	0.0797	0.508	204.961	304293.0	4.0	[3.0,58.0,2012.0,...]
4	54.0	2012	0.43	0.791	6	-5.419	0	0.0302	0.0726	0.0193	0.11	0.217	171.864	244320.0	4.0	[4.0,54.0,2012.0,...]
5	48.0	2012	0.566	0.57	2	-6.42	1	0.0329	0.688	1.73E-6	0.0943	0.96	83.403	166240.0	4.0	[5.0,48.0,2012.0,...]
6	48.0	2012	0.575	0.606	9	-8.197	1	0.03	0.0119	0.0	0.0675	0.364	121.083	152307.0	4.0	[6.0,48.0,2012.0,...]
7	45.0	2012	0.586	0.423	7	-7.459	1	0.0261	0.252	5.83E-6	0.0976	0.318	138.133	232373.0	4.0	[7.0,45.0,2012.0,...]
8	44.0	2012	0.65	0.628	7	-7.16	1	0.0232	0.0483	0.0	0.119	0.7	84.141	235080.0	4.0	[8.0,44.0,2012.0,...]
9	58.0	2012	0.619	0.28	8	-10.238	0	0.0317	0.73	0.0	0.103	0.292	129.948	250063.0	4.0	[9.0,58.0,2012.0,...]
10	45.0	2012	0.657	0.43	1	-10.202	1	0.0314	0.534	2.38E-4	0.12	0.335	91.967	236379.0	4.0	[10.0,45.0,2012.0,...]
11	45.0	2012	0.655	0.692	5	-6.217	1	0.0326	0.568	2.34E-5	0.119	0.547	121.455	234270.0	4.0	[11.0,45.0,2012.0,...]
12	55.0	2012	0.439	0.207	1	-9.573	1	0.0297	0.608	0.0	0.186	0.264	136.514	276147.0	4.0	[12.0,55.0,2012.0,...]
13	40.0	2012	0.591	0.647	4	-8.34	1	0.0277	0.067	6.57E-5	0.231	0.678	79.68	190752.0	4.0	[13.0,40.0,2012.0,...]
14	44.0	2012	0.45	0.713	6	-7.503	0	0.0386	0.157	0.0	0.092	0.36	74.71	283587.0	4.0	[14.0,44.0,2012.0,...]
15	41.0	2012	0.497	0.277	4	-11.382	0	0.0373	0.846	4.77E-6	0.111	0.179	81.367	222560.0	4.0	[15.0,41.0,2012.0,...]
16	57.0	2012	0.439	0.265	2	-10.72	1	0.0343	0.736	0.0	0.093	0.382	140.494	247002.0	4.0	[16.0,57.0,2012.0,...]
17	39.0	2012	0.499	0.614	7	-8.175	1	0.0621	0.151	3.52E-6	0.0555	0.605	89.038	276973.0	4.0	[17.0,39.0,2012.0,...]
18	49.0	2012	0.713	0.824	3	-7.168	1	0.0393	0.106	8.03E-5	0.13	0.696	120.028	207240.0	4.0	[18.0,49.0,2012.0,...]
19	39.0	2012	0.275	0.216	2	-14.504	1	0.0493	0.896	0.0	0.231	0.0551	95.421	324333.0	5.0	[19.0,39.0,2012.0,...]

سپس داده هارا نرمال سازی می کنیم تا مقادیر feature ها در یک محدوده و خیلی پرت نباشند :

```
[10] # نرمال سازی داده ها
scaler = StandardScaler(inputCol="features", outputCol="scaledFeatures")
scaler_model = scaler.fit(spotifydf_features)
spotifydf_scaled = scaler_model.transform(spotifydf_features)
```

_c0	popularity	year	danceability	energy	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	duration_ms	time_signature	features	scaledFeatures
0	68.0	2012	0.483	0.303	4	-10.058	1	0.0429	0.694	0.0	0.115	0.139	133.406	240166.0	3.0	[0.0,68.0,2012.0,...]	[0.0,4.2799025845...]
1	50.0	2012	0.572	0.454	3	-10.286	1	0.0258	0.477	1.37E-5	0.0974	0.515	140.182	216387.0	4.0	[1.0,50.0,2012.0,...]	[2.33334273591497...]
2	57.0	2012	0.409	0.234	3	-13.711	1	0.0323	0.338	5.0E-5	0.0895	0.145	139.832	158960.0	4.0	[2.0,57.0,2012.0,...]	[4.66668547182994...]
3	58.0	2012	0.392	0.251	10	-9.845	1	0.0363	0.807	0.0	0.0797	0.508	204.961	304293.0	4.0	[3.0,58.0,2012.0,...]	[7.00002820774491...]
4	54.0	2012	0.43	0.791	6	-5.419	0	0.0302	0.0726	0.0193	0.11	0.217	171.864	244320.0	4.0	[4.0,54.0,2012.0,...]	[9.33337094365989...]
5	48.0	2012	0.566	0.57	2	-6.42	1	0.0329	0.688	1.73E-6	0.0943	0.96	83.403	166240.0	4.0	[5.0,48.0,2012.0,...]	[1.16667136795748...]
6	48.0	2012	0.575	0.606	9	-8.197	1	0.03	0.0119	0.0	0.0675	0.364	121.083	152307.0	4.0	[6.0,48.0,2012.0,...]	[1.40000564154898...]
7	45.0	2012	0.586	0.423	7	-7.459	1	0.0261	0.252	5.83E-6	0.0976	0.318	138.133	232373.0	4.0	[7.0,45.0,2012.0,...]	[1.63333991514048...]
8	44.0	2012	0.65	0.628	7	-7.16	1	0.0232	0.0483	0.0	0.119	0.7	84.141	235080.0	4.0	[8.0,44.0,2012.0,...]	[1.86667418873197...]
9	58.0	2012	0.619	0.28	8	-10.238	0	0.0317	0.73	0.0	0.103	0.292	129.948	250063.0	4.0	[9.0,58.0,2012.0,...]	[2.10000846232347...]

پس از انجام نرمال سازی داده ها ، خوشه بندی را با استفاده از الگوریتم های مختلف شروع می کنیم .

اولین الگوریتم ، الگوریتم خوشه بندی k-means هستش که به صورت قطعه کد زیر آن را اجرا کردیم :

```
from pyspark.ml.evaluation import ClusteringEvaluator

# الگوریتم خوشه بندی k-means و رسم نمودار
cost = []
k_values = range(2, 11)
silhouette_scores = []

for k in k_values:
    kmeans = KMeans(featuresCol="scaledFeatures", k=k, seed=1)
    model = kmeans.fit(spotifydf_scaled)
    cost.append(model.summary.trainingCost)

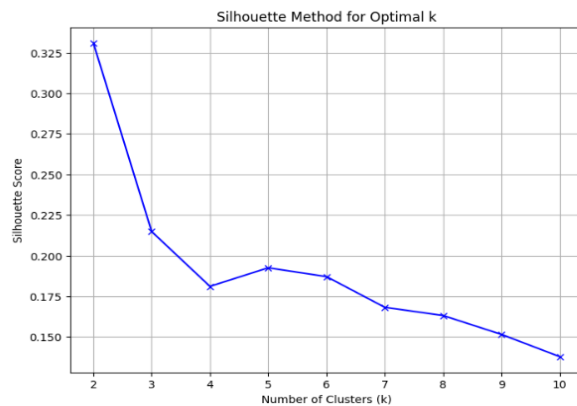
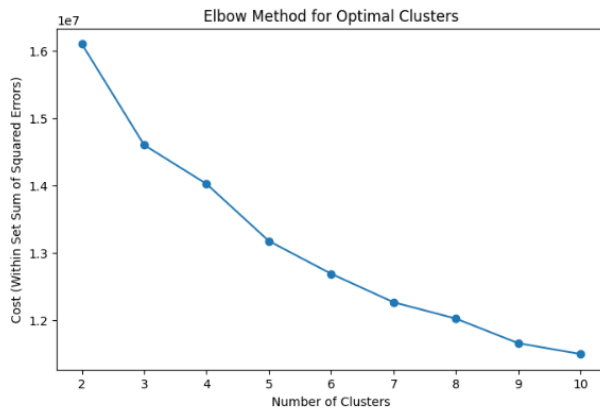
    predictions = model.transform(spotifydf_scaled)
    evaluator = ClusteringEvaluator(featuresCol="scaledFeatures", metricName="silhouette", distanceMeasure="squaredEuclidean")
    silhouette = evaluator.evaluate(predictions)
    silhouette_scores.append(silhouette)

# رسم نمودار
plt.figure(figsize=(8, 5))
plt.plot(k_values, cost, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("Cost (Within Set Sum of Squared Errors)")
plt.title("Elbow Method for Optimal Clusters")
plt.show()

# 7. Silhouette رسم نمودار
plt.figure(figsize=(8, 6))
plt.plot(k_values, silhouette_scores, 'bx-')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Method for Optimal k')
plt.grid()
plt.show()

# به دست آوردن تعداد ژانرهای واقعی
genre_count = spotifydf.select("genre").distinct().count()
print(f"Number of unique genres: {genre_count}")
```

نمودار elbow و Silhouette به دست آمده از طریق این الگوریتم به صورت زیر است :



الگوریتم بعدی الگوریتم Gaussian Mixture Model (GMM) است. الگوریتم GMM برای خوشه‌بندی توزیع گوسی استفاده می‌شود و برخلاف K-Means می‌تواند خوشه‌هایی با اندازه‌های متفاوت و شکل‌های بیضوی پیدا کند، که به صورت زیر آن را اجرا کردیم :

```
from pyspark.ml.clustering import GaussianMixture
import matplotlib.pyplot as plt

# Gaussian Mixture Model اجرای
cost = []
k_values = range(2, 11)
silhouette_scores = []

for k in k_values:
    gmm = GaussianMixture(featuresCol="scaledFeatures", k=k, seed=1)
    model = gmm.fit(spotifydf_scaled)
    log_likelihood = model.summary.logLikelihood
    cost.append(log_likelihood)

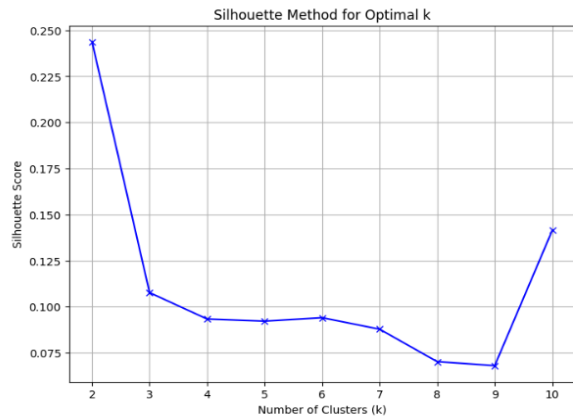
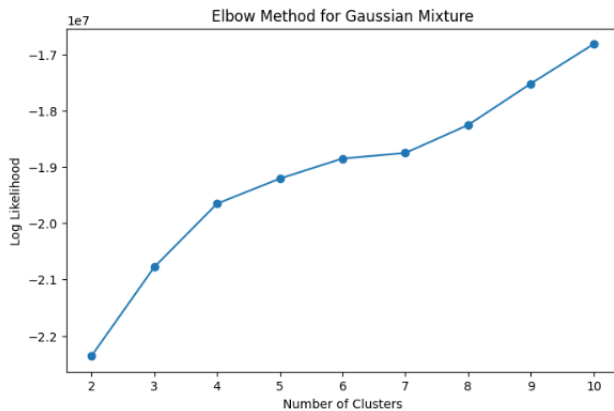
    predictions = model.transform(spotifydf_scaled)
    evaluator = ClusteringEvaluator(featuresCol="scaledFeatures", metricName="silhouette", distanceMeasure="squaredEuclidean")
    silhouette = evaluator.evaluate(predictions)
    silhouette_scores.append(silhouette)

# elbow نمودار رسم
plt.figure(figsize=(8, 5))
plt.plot(k_values, cost, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("Log Likelihood")
plt.title("Elbow Method for Gaussian Mixture")
plt.show()

# 7. Silhouette نمودار رسم
plt.figure(figsize=(8, 6))
plt.plot(k_values, silhouette_scores, 'bx-')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.title('Silhouette Method for Optimal k')
plt.grid()
plt.show()
```

از مقدار Log Likelihood به عنوان معیاری برای تعیین کیفیت خوشه‌بندی استفاده می‌کنیم. هر چه Log Likelihood بزرگتر باشد، مدل بهتر است.

نتایج به دست آمده به صورت ذیل است :



الگوریتم بعدی ، الگوریتم DBSCAN است . در پای اسپارک ، نسخه اصلی کتابخانه به صورت پیش فرض از الگوریتم DBSCAN پشتیبانی نمی کند ، زیرا این الگوریتم هنوز به طور رسمی در پای اسپارک پیاده سازی نشده است . علی رغم این موضوع سعی کردیم از توابع پای اسپارک استفاده کنیم تا این الگوریتم را پیاده سازی کنیم . یکی از روش هایی که امتحان کردیم به صورت زیر بود :

```
[ ] from pyspark.sql import SparkSession

# ایجاد SparkSession با افزودن dbscan-on-spark
spark = SparkSession.builder \
    .appName("DBSCAN Example") \
    .config("spark.jars.packages", "org.alitouka:spark-dbscan_2.11:0.0.5") \
    .getOrCreate()

[ ] from dbscan import DBSCAN

# اجرای DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5).setFeaturesCol("scaledFeatures")
model = dbscan.fit(spotifydf_scaled)

# مشاهده خوشه ها
clusters = model.transform(spotifydf_scaled)
clusters.show()
```

این روش پیشنهادی چت جی پی تی بود که کار نکرد و با خطای زیر مواجه شدیم :

```
ModuleNotFoundError                               Traceback (most recent call last)
<ipython-input-21-b5a2091eec87> in <cell line: 1>()
----> 1 from dbscan import DBSCAN
      2
      3 # اجرای DBSCAN
      4 dbscan = DBSCAN(eps=0.5, min_samples=5).setFeaturesCol("scaledFeatures")
      5 model = dbscan.fit(spotifydf_scaled)

ModuleNotFoundError: No module named 'dbscan'

-----
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
-----
```

لازم به ذکر است که قبل از زدن این کد چندین پکیج را نصب کردیم که باز موفقیت آمیز نبود .

به عنوان یک روش دیگر ، از کتابخانه ی Third-party (مثل MLlib-local یا spark-DBSCAN) هم استفاده کردیم . Spark-DBSCAN یک افزونه است که DBSCAN را در Spark پیاده سازی می کند. این روش هم جواب نداد و به خطای قبلی دوباره برخورد کردیم .

از روش گفته شده در سایت گیت هاب هم استفاده کردیم که موقع اجرای آن به خطای کوچکی برخورد می کردیم که در صورت رفع آن به خطای جدید دیگری برخورد می کردیم و از اجرای این روش هم صرف نظر کردیم .

به عنوان راه حل این موضوع ، از پای اسپارک و اجرای DBSCAN با Scikit-learn در DataFrame های کوچک استفاده کردیم . اگر داده های ما نیاز به پردازش موازی اسپارک نداشته باشد ، می توانیم داده ها را به Pandas منتقل کرده و DBSCAN را با Scikit-learn اجرا کنیم که این روش جواب داد و به صورت قطعه کد زیر آن را اجرا کردیم :

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Pandas به داده ها
data_pandas = sampled_data.select("scaledFeatures").toPandas()
features = data_pandas["scaledFeatures"].apply(lambda x: x.toArray()).tolist()

# استاندارد سازی داده ها
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)

# eps و min_samples محدوده پارامترهای
eps_values = [0.2, 0.4, 0.6, 0.8, 1.0]
min_samples_values = [2, 4, 6]

# نگهداری نتایج
results = []

# eps و min_samples حلقه روی پارامترهای
for eps in eps_values:
    for min_samples in min_samples_values:
        dbscan = DBSCAN(eps=eps, min_samples=min_samples)
        labels = dbscan.fit_predict(scaled_features)

        # تعداد خوشه ها
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0) # حذف نویز (-1)
        n_noise = list(labels).count(-1)

        # محاسبه Silhouette Score (در صورت وجود بیش از یک خوشه)
        if n_clusters > 1:
            silhouette = silhouette_score(scaled_features, labels)
        else:
            silhouette = -1 # مقدار غیر معتبر

        results.append((eps, min_samples, n_clusters, silhouette, n_noise))

# DataFrame به نتایج تبدیل
results_df = pd.DataFrame(results, columns=["eps", "min_samples", "n_clusters", "silhouette_score", "n_noise"])

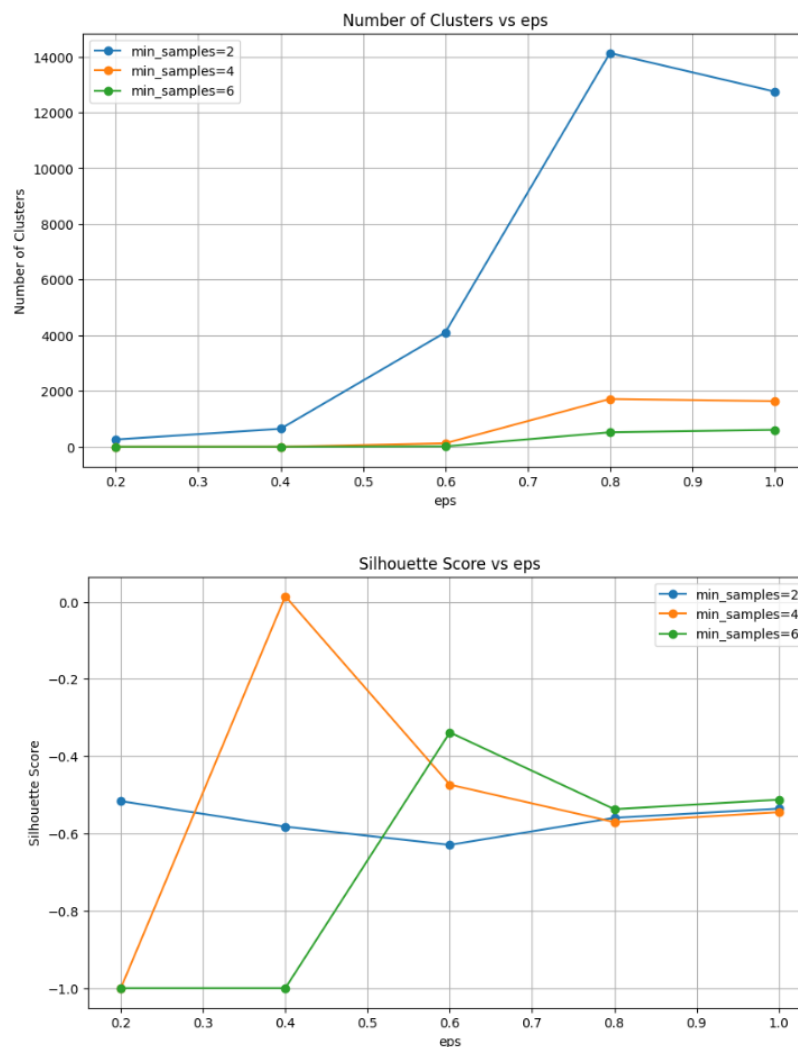
# رسم نمودارها
# 1. نمودار تعداد خوشه ها
plt.figure(figsize=(10, 6))
for min_samples in min_samples_values:
    subset = results_df[results_df["min_samples"] == min_samples]
    plt.plot(subset["eps"], subset["n_clusters"], marker='o', label=f'min_samples={min_samples}')
plt.title('Number of Clusters vs eps')
plt.xlabel('eps')
plt.ylabel('Number of Clusters')
plt.legend()
plt.grid(True)
plt.show()

# 2. Silhouette Score نمودار
plt.figure(figsize=(10, 6))
for min_samples in min_samples_values:
    subset = results_df[results_df["min_samples"] == min_samples]
    plt.plot(subset["eps"], subset["silhouette_score"], marker='o', label=f'min_samples={min_samples}')
plt.title('Silhouette Score vs eps')
plt.xlabel('eps')
plt.ylabel('Silhouette Score')
plt.legend()
plt.grid(True)
plt.show()
```

برای اجرای الگوریتم DBSCAN ، تعیین تعداد خوشه‌های بهینه به صورت مستقیم مشابه KMeans نیست، زیرا DBSCAN نیازمند تعداد خوشه‌های اولیه (k) نیست. در عوض، پارامترهای **eps** (شعاع همسایگی) و **minPoints** تأثیر زیادی بر نتایج خوشه‌بندی دارند. ما می‌توانیم کیفیت خوشه‌بندی را با Silhouette Score ارزیابی کنیم و مقدار بهینه پارامترها را بیابیم. سپس با استفاده از تعداد خوشه‌های نهایی، نمودارها را ترسیم می‌کنیم.

این کد را روی ۲۵ درصد داده‌ها اجرا کردیم ، زیرا این الگوریتم برای داده‌های با ابعاد بالا محاسبات سنگینی دارد. برای تمام داده‌ها امتحان کردیم که نزدیک به ۱۲ ساعت زمان برد و در نهایت نتیجه هم نداد و مدت زمان جلسه ی رایگان گوگل کولب تمام شد .

نتایج نهایی اجرای این الگوریتم به صورت زیر است :



الگوریتم بعدی ، الگوریتم Bisecting-KMeans است . این الگوریتم را با قطعه کد زیر اجرا کردیم :

```
from pyspark.ml.clustering import BisectingKMeans
from pyspark.ml.evaluation import ClusteringEvaluator
from pyspark.sql import SparkSession
import matplotlib.pyplot as plt

# ستون ویژگی ها
features_col = "scaledFeatures"

# لیست مقادیر k
k_values = range(2, 11) # از 2 تا 10 خوشه

# نگهداری مقادیر هزینه
costs = []
silhouette_scores = []

# حلقه روی مقادیر مختلف k
for k in k_values:
    bkm = BisectingKMeans().setK(k).setSeed(1).setFeaturesCol(features_col)
    model = bkm.fit(spotifydf_scaled)

    # هزینه خوشه‌بندی (Within Set Sum of Squared Errors)
    cost = model.computeCost(spotifydf_scaled)
    costs.append(cost)

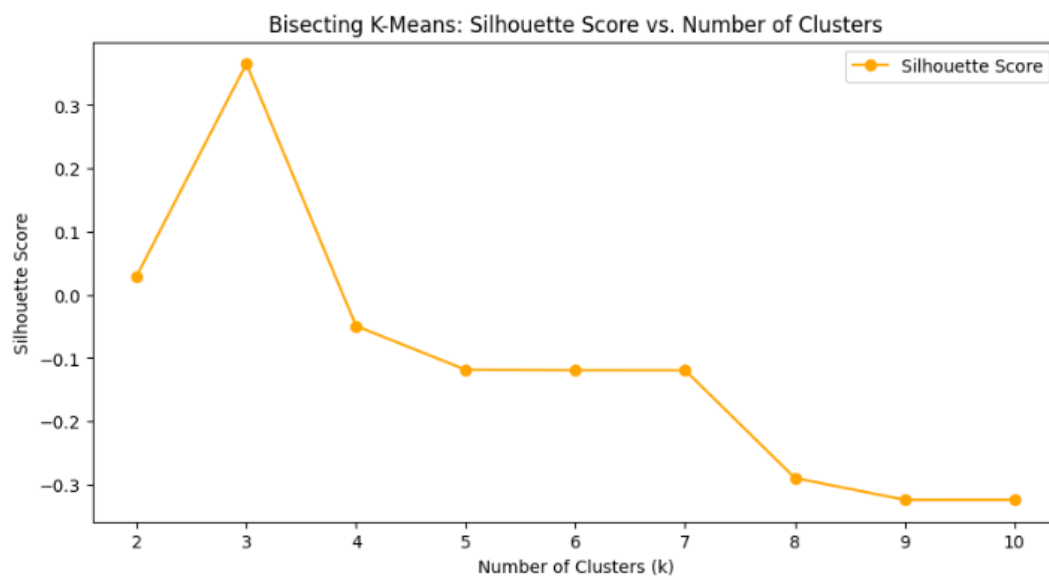
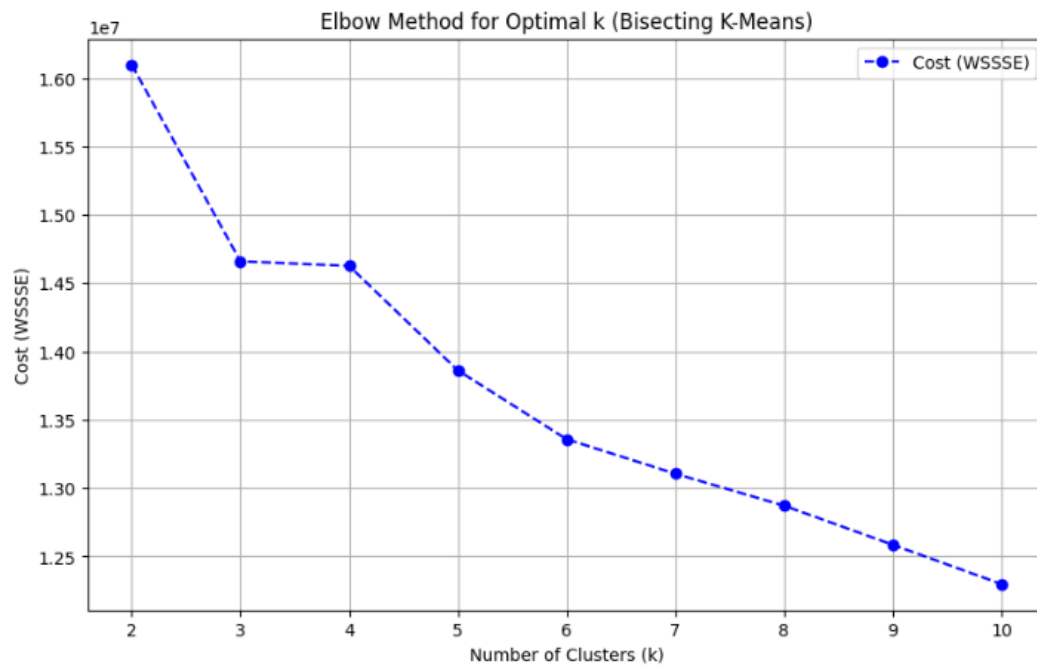
    # ارزیابی با silhouette score
    predictions = model.transform(spotifydf_scaled)
    evaluator = ClusteringEvaluator()
    silhouette = evaluator.evaluate(predictions)
    silhouette_scores.append((k, silhouette))

# نمایش نتایج
print("Costs:", costs)
print("Silhouette Scores:", silhouette_scores)

# رسم نمودار Elbow
plt.figure(figsize=(10, 6))
plt.plot(k_values, costs, marker='o', linestyle='--', color='b', label='Cost (WSSSE)')
plt.title('Elbow Method for Optimal k (Bisecting K-Means)')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Cost (WSSSE)')
plt.xticks(k_values)
plt.grid(True)
plt.legend()
plt.show()

# رسم نمودار silhouette score
k_vals, sil_vals = zip(*silhouette_scores)
plt.figure(figsize=(10, 5))
plt.plot(k_vals, sil_vals, marker='o', color='orange', label='Silhouette Score')
plt.title('Bisecting K-Means: Silhouette Score vs. Number of Clusters')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.legend()
plt.show()
```


نتایج به دست آمده به صورت زیر می باشد :



مقایسه ی عملکرد الگوریتم های مختلف :

۱. k-means :

- روش مبتنی بر خوشه‌بندی مرکزی که خوشه‌ها را بر اساس فاصله اقلیدسی تعریف می‌کند.
- سریع و ساده .
- مناسب برای داده‌های بزرگ.
- عملکرد خوبی برای خوشه‌های کروی و داده‌های همگن دارد.
- سرعت بالا .

۲. Bisecting K-Means :

- نسخه بهبودیافته K-Means که خوشه‌بندی را به صورت بازگشتی (دو به دو) انجام می‌دهد. ابتدا داده‌ها به دو خوشه تقسیم می‌شوند، سپس فرایند تقسیم برای هر خوشه ادامه می‌یابد.
- سرعت آن مشابه K-Means اما بسته به تعداد تقسیمات، می‌تواند کندتر باشد.
- مناسب برای داده‌های متوسط تا بزرگ.
- دقت آن اغلب بهتر از K-Means است، زیرا با چندین تقسیم‌بندی امکان کاهش خطا وجود دارد.

۳. DBSCAN (Density-Based Spatial Clustering of Applications with Noise).

- خوشه‌بندی بر اساس تراکم نقاط.
- پیچیدگی زمانی دارد و برای داده‌های با ابعاد زیاد بسیار کند است.
- دقت عالی برای داده‌های نامتقارن و خوشه‌های با اشکال پیچیده.

۴. Gaussian Mixture Model (GMM)

- مدل مبتنی بر توزیع گوسی که داده‌ها را به صورت احتمالاتی در خوشه‌ها تقسیم می‌کند.
- هر خوشه یک توزیع گوسی مجزا دارد.
- پیچیدگی زمانی بیشتر از K-Means.
- دقت بالاتر از K-Means برای داده‌های پیچیده.

مقایسه نهایی :

معیار	k-means	Bisecting K-Means	DBSCAN	GMM
سرعت	بسیار سریع	سریع تر از GMM	کند	متوسط
دقت	متوسط	بهتر از k-means	بالا برای داده های پیچیده	بالا